

Using MongoDB Compass to cross-verify

Go to MongoDB Compass. You will see the following screen.

Here by default it asks you to connect to the above URI. This is the same URI we configured to connect in our express app, so let's just hit connect. And navigate to users collection as shown below.

As you can see the user is successfully added to our mongo database. We can also see that password is hashed and saved in database. Here `_id` is a unique key created by default for every record we add to the collection.

This is all about creating a record in mongo using NodeJS. Let's move to read operation.

Reading documents in MongoDB using NodeJS

Let's create a API for reading records from database. For reading we send a GET request to the API with request parameter to identify the record. Copy the below code to `index.js` in route folder.

```
router.get('/read/:username', async (req,res)=>{
  try{
    const username = req.params.username
    const result = await User.findOne({username});
    if(result)
      return res.send(result)
    else
      return res.status(404).send(`${username} does not exist`)
  }catch(err){
    res.status(500).send(err)
  }
})
```

Here we have created a `'/read/:username' GET` endpoint, where we are passing the username to find the record associated with it. First we are retrieving the username from request parameter then using a **findOne** method of mongoose to retrieve the record associated with it from database and passing it back as

response. If user is not found we are sending 404 back in response with a message. In case of any other error, we are doing the same as we did above.

FindOne Vs Find

You will notice we are using **findOne** method, which as the name suggests will fetch only one record, also we have a unique constraint on username which means we will only have a single record for every username in database.

We can use **find** as well instead of **findOne**, it will give the same result. Only difference is that we use **find** to get one or more records matching the condition.

Let's see we want to fetch all the users present in our database. Code will be as follows,

```
//Reading multiple records
router.get('/read', async (req,res)=>{
  try{
    const result = await User.find();
    return res.send(result)
  }catch(err){
    res.status(500).send(err)
  }
})
```

Here we just have '/read' endpoint without any request parameter as we want to read the entire collection. Similar to **findOne**, here we have used **find** method on User to retrieve all the documents in User collection.

Testing Read APIs

Let's test read APIs using rest client.

We see here that when we are hitting 'http://localhost:8000/read/saydo' where saydo is the username we are trying to fetch record for, and we are getting the entire record associated with that username.

Here we are fetching all the records in database not just any particular record. And we get the list of all records available in database.

Updating documents in MongoDB using NodeJS

For updating let's take an example of updating nickname in a particular record. Code will be as follows

```
router.put('/update', async (req,res)=>{
  try{
    const username = req.body.username
    const newNickName = req.body.nickname
    const result = await User.findOneAndUpdate(
      { username: username },
      {
        $set: {
          nickname: newNickName,
        }
      },
      { new: true }
    );
    res.send(result)
  }catch(err){
    res.status(500).json(err.message)
  }
})
```

Here we are creating a **PUT** endpoint which will receive username against which we have to update the nickname. New nickname also we will get in request body. Then we are using findOneAndUpdate method of mongoose which will find the particular record in our case matching with the username we are providing then updates the nickname with new nickname. You will notice a key new set to true, that is used, so the query will return the updated new document.

If a matching document does not contain the nickname field, then the query will add the field and update its value.

Difference between findOneAndUpdate, updateOne, update and updateMany

We just used **findOneAndUpdate** and know how it works, similar to **findOneAndUpdate** we also have **updateOne** method, It also updates only the top most document in a collection with matching filter. Only difference between **findOneAndUpdate** and **updateOne** is that, **updateOne** does not return a document whereas **findOneAndUpdate** does.

UpdateMany will update all the documents with the matching condition. And **update** method by default will update only one document but if we include option { **multi : true** } then it will update all the documents matching the filter.

Testing Update API

Lets test the update API in rest client by changing the nickname for the username we created above.

Here we are hitting the update API with username and nickname in request body. And we get the updated document in response.

Let's check in Mongo if the document is updated.

As you can see, the document is updated with new nickname.

Deleting documents in MongoDB using NodeJS

Let's try to delete the user we created above. Code will be as follows.

```
router.delete('/delete', async (req,res)=>{
  try{
    const username = req.body.username
    const result = await User.deleteOne({username});
    res.send(result)
  }catch(err){
    res.status(500).json(err.message)
  }
})
```

Here we have created a **DELETE** endpoint which will receive username in request body, and we will delete the entire document matching that username.

Here we have used **deleteOne** method which only delete the first matching document in database.

deleteOne vs deleteMany

As the name suggests deleteOne deletes only single document matching the filter whereas deleteMany will delete all the documents matching the filter.

Testing Delete API

Let's test delete API by deleting the document associated with the username we created above.

Here we are triggering the delete API with username in request body and we get the success response with count of deleted documents.

This covers all the CRUD operations.